

Test plan for: Co-App

Note that you can refine your testing plan as the project development goes. Keep the change log as follow:

ChangeLog

Version	Change Date	By	Description
1.0	March 5th	All members	Initial testing plan Cover Feature: 1,2,4

1 Introduction

1.1 Scope

This is our testing scope for Sprint 2:

1. Feature 1: User authentication
 - a. Create account
 - b. Log in
 - c. Log out
 - d. Change password
2. Feature 2: Log Job Applications
 - a. Log new application
 - b. Delete entry
 - c. Update Status
 - d. Edit Application Details
3. Feature 4: Company Wiki (“Rate my Co-op”)
 - a. Add Company profile
 - b. Write Review
 - c. Read Reviews

1.2 Roles and Responsibilities

Each team member takes on one or more of the following roles:

- QA Analyst
- Test Manager
- Backend Developers
- Frontend Developers
- Database Manager
- DevOps

Name	Net ID	GitHub username	Role
Bao Ngo	ngot1	benjaminNngo	Backend Developer Test Manager DevOps - CI/CD
Aidan McLeod	mcleo16	ACM02	Frontend Developer QA Analyst DevOps - CI/CD
Nikolai Barlaan	barlaann	yomi-adt	Frontend dev QA Analyst
Eric Hodgson	hodgsone	EricHodgson	Backend Developer Test Manager Database Manager
Kate Lacerna	lacernak	lacernakate	Frontend Developer QA Analyst
Geet	loombag	gloox	Backend Developer Test Manager DevOps - Deployment

Role Details:

1. A Front-end Developer is responsible for:
 - Create UI components
 - Perform API calls to endpoints according to the [API documentation](#) for each feature.
 - Perform code review for front-end PRs
 - Design a sequential diagram up to API calls
2. A back-end developer is responsible for
 - Maintain the database, develop business services and expose endpoint controllers
 - Design [API document](#)
 - Perform code review for back-end PRs

- Add components from the backend to the sequential diagram
- 3. QA Analyst is responsible for:
 - Test the functionality of the front-end components
 - Perform acceptance tests
- 4. Test manager is responsible for:
 - Manages and implements tests to achieve 100% code coverage for the backend
 - Implement integration tests
 - Implement load tests.
- 5. DevOps is responsible for either:
 - Create and maintain CI/CD
 - Deploy application
- 6. Database Manager
 - Create and host MongoDB database instances for dev and prod environments

2 Test Methodology

2.1 Test level

Test levels define the Types of Testing to be executed on the Application Under Test (AUT). In this course, unit testing, integration testing, acceptance testing, regression testing, and load testing are mandatory.

Feature 1: User authentication

Unit testing - Automated with regression testing

(JwtAuthFilter) doFilterInternal_whenEverythingSuccess

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/config/JwtAuthFilterTest.java#L68C3-L86C4>

Check if the JWT token is parsed successfully

(AuthController) createAccount_whenEverythingSuccess_expect200Response

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/AuthControllerTest.java#L178C3-L195C4>

Check the register endpoint with the mock all service and mock MVC - successful case

(UserController) updatePassword_whenIncorrectOldPassword_expect401

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller>

[/UserControllerTest.java#L89C3-L104C4](#)

Check the update password endpoint with the mock all service and mock MVC - fail case due to incorrect old password

(UserMode constructor) constructor_whenNoIDProvided_expectAutoGenerateID

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/UserModelTest.java#L95C3-L105C4>

Test UserModel's constructor when no ID is provided and expect the ID to be auto-generated

(UserRepository) findUserModelByEmail_whenFindByJohnEmail_expectJohnUserModel

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/UserRepositoryTests.java#L62C3-L73C4>

Test UserRepository with a MongoDB instance in the test container. Test to query a user with email.

(AuthService) createUser_whenNewEmailIsUsed_expectEmailSentWith

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/AuthServiceTest.java#L108C3-L133C1>

Test the business logic of registering a new account.

(EmailService) sendEmail_whenSendSuccessfully_expectNoException

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/EmailServiceTest.java#L87C3-L98C4>

Test the business logic of sending an email.

(UserService) updateUserPassword_whenOldPasswordMatch_expectNoException

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/UserServiceTest.java#L135C3-L145C1>

Test the business logic of updating the password.

Integration testing

(Feature 1 + Feature 2)

applicationFlow_whenUserLogsInAndCreatesUpdatesDeletesApplicationAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/ApplicationLifecycleCrossFeatureIntegrationTest.java#L88

Tests the full lifecycle of a job application: create, update, and delete, integration testing with the Authentication, Application Management, and Company features together.

(Feature 1 + Feature 4)

reviewFlow_whenUserLogsInAndPostsReviewAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L84

Tests that an authenticated user can post a review for a company, integrating Authentication, Review Management, and Company Profile features, verifying the review is persisted and the company's average rating is updated correctly.

(Feature 1 + Feature 4)

reviewFlow_whenUserLogsInAndCreatesDuplicatesAndUpdatesReviewAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L154C7-L154C100

Tests that an authenticated user cannot submit duplicate reviews but can update an existing one, integrating Authentication, Review Management, and Company Profile features, verifying duplicate rejection, correct persistence, and accurate average rating recalculation.

(Feature 1 + Feature 2 + Feature 4)

wikiFlow_whenUserLogsInAndSearchesPaginatesReviewsAndAppliesAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/SearchPaginationCrossFeatureIntegrationTest.java#L95C7-L95C99

Tests that an authenticated user can search and paginate companies, leave a review, and create a job application from the company profile, integrating Authentication, Company Search, Review Management, and Application Management features together, verifying that all data is correctly persisted and consistent across features.

(Feature 1 + Feature 2 + Feature 4)

wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L96C7-L96C104

Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search, and Application Management features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

1. Given that I'm on the log-in screen and want to make a new account, when I click the "create new account" button, then I am greeted with a new interface to submit an email and password.
2. Given that I'm looking at the interface to submit an email and password, when I submit a valid email that is not already associated with an account and a "valid" password, then an email will be sent to the email address I submitted that contains a verification code and I'm greeted with a new interface prompting me for that code.
 - a. "Valid" password criteria: Must not be empty (must contain at least one non-whitespace character). Cannot start with a whitespace character. Cannot end with a whitespace character
3. Given that I'm looking at the interface to submit an email and password, when I submit an invalid email, I will be shown a message saying that the email I submitted was invalid.
4. Given that I'm looking at the interface to submit an email and password, when I submit an email already associated with an account, then I will be shown a message saying that I must choose a different email.
5. Given that I'm looking at the interface to submit an email and password, when I submit an invalid password, I will be shown a message saying that the password I submitted was invalid.
 - a. "Valid" password criteria: Must not be empty (must contain at least one non-whitespace character). Cannot start with a whitespace character. Cannot end with a whitespace character
6. Given that I have access to that email address and see the email containing the verification code, when I take that code and input it into the new interface, then I will have successfully made an account and can log in using the email/password combination that I put in.
7. Given that I have access to that email address and don't see the email containing the verification code, when I want to receive a new code because I didn't receive it, then I can press the "re-send code" button on the screen to receive a new code.
8. Given that I'm logged out, have an account created, and on the log-in screen, when I type in my email and password correctly, then I will be logged in, be able to access my data, and perform actions on the application.
9. Given that I'm logged-out, have an account created, and on the log-in screen, when I type my email/password incorrectly, then I will be prompted to try again, my data will be inaccessible, and I will not be able to any actions on the application
10. Given that I'm logged-in and want to log-out, when I click the "log-out" button then I will lose access to my data and will be unable to perform the app's functionalities

11. Given that I am logged in and want to change my password, when I click the “edit profile” button at the top of the screen, then I will see a page that contains a “change password” button.
12. Given that I’m on the “edit profile” screen, when I click on the “change password” button, then I will see an interface with fields to input my current password and my new password
13. Given that I see the “change password” interface, when I input my current password correctly with a new, valid password, then the password I use to log into this account changes to the new password that I sent.
 - a. “Valid” password criteria: Must not be empty (must contain atleast one non-whitespace character). Cannot start with a whitespace character. Cannot end with a whitespace character
14. Given that I see the “change password” interface, when I input my current password incorrectly, then I will see a message saying that I put in an incorrect password
15. Given that I see the “change password” interface, when I input my current password correctly but input an invalid password as my new password, then I will see a message saying that my new password is invalid
16. Given that I see the “change password” interface, when I input my current password correctly but put in that same password as my new password, then I will see a message saying that my new password can not be my old password
17. Given that I’ve successfully changed my password, when I log-in with my new password then I will be granted access to my account and be logged-in
18. Given that I’ve successfully changed my password, when I log-in with my old password then I will be denied access and see a message saying “wrong password”

Feature 2: Log Job Application

Unit testing - Automated with regression testing

ApplicationModelTest) validate_whenAllFieldsValid_expectNoViolations

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/ApplicationModelTest.java#L68-L72> Verify that the entity validation logic passes when all required fields are correctly set.

(ApplicationModelTest) validate_whenUserIdNull_expectViolation

<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/ApplicationModelTest.java#L74-L80> Verify that the entity validation logic correctly catches null User ID constraints.

(ApplicationRepositoryTest) findByUserId_whenUserHasTwoApps_expectReturnTwo
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/ApplicationRepositoryTest.java#L72-L80> Test the repository integration with a MongoDB test container to ensure custom query methods work correctly.

(ApplicationServiceTest) createApplication_whenValid_expectSuccess
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L96-L114> Test the main integration flow for creating an application using the database container.

(ApplicationServiceTest) updateApplication_whenNotOwner_expectUnauthorized
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L194-L211> Test the business logic security to ensure users cannot update applications they do not own.

(ApplicationServiceTest) createApplication_whenDatabaseFails_expectServiceFailException
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L320-L345> Test the unit logic using mocks to ensure graceful handling of database runtime exceptions.

(ApplicationControllerTest) createApplication_whenValid_expect201AndApplication
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L105-L145> Check the create endpoint with MockMvc - successful case (Happy Path).

(ApplicationControllerTest) updateApplication_whenNotOwned_expect403
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L337-L362> Check the update endpoint to ensure correct HTTP 403 Forbidden status when authorization fails.

(ApplicationControllerTest) getApplications_whenValid_expect200AndApplicationList
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L590-L603> Check the get all applications endpoint to verify the JSON response structure and status.

Integration testing

(Feature 1 + Feature 2)
applicationFlow_whenUserLogsInAndCreatesUpdatesDeletesApplicationAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/ApplicationLifecycleCrossFeatureIntegrationTest.java#L88

Tests the full lifecycle of a job application: create, update, and delete, integration testing with the Authentication, Application Management, and Company features together.

(Feature 1 + Feature 2 + Feature 4)

wikiFlow_whenUserLogsInAndSearchesPaginatesReviewsAndAppliesAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/SearchPaginationCrossFeatureIntegrationTest.java#L95C7-L95C99

Tests that an authenticated user can search and paginate companies, leave a review, and create a job application from the company profile, integrating Authentication, Company Search, Review Management, and Application Management features together, verifying that all data is correctly persisted and consistent across features.

(Feature 1 + Feature 2 + Feature 4)

wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L96C7-L96C104

Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search, and Application Management features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

1. Given that I'm on a page for showing all the applications I've logged, when I click on a button that says "add new application", then I will see a new interface that will allow me to input a company name, job title, deadline date, and a link
2. Given that I see the interface for submitting the new application, when I input these fields and click a submit button, then a new application will be added to my list with these details
3. Given that I see the interface for submitting the new application, when I input an empty company name or job title ("empty" as in I leave it blank or input a name/job title that is only empty characters) then I will be shown a message saying that I the name/job title is invalid and I will not be allowed to press submit

4. Given that I see the interface for submitting the new application, when I don't select a deadline date then I will see a message saying to select a date and will not be allowed to press the submit button
5. Given that I successfully added a new application, when I return to the page showing the applications I've logged, then I can see the new application within the list.

Feature 4: Company Wiki ("Rate my co-op")

Unit testing - Automated with regression testing

1. (review controller) createReview_whenValid_expect201AndReview
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ReviewControllerTest.java#L78>
Test creating a new review endpoint with mock services and mock MVC
2. (company controller) createCompany_whenAlreadyExists_expect409
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/CompanyControllerTest.java#L332>
Test creating a new company endpoint with mock services and mock MVC
3. (company service)
createCompany_whenCompanyExistsCaseInsensitive_expectException
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/CompanyServiceTest.java#L90>
Test creating a new company business logic - when the company already exist
4. (company service) updateAvgRating_whenMultipleReviews_expectCorrectAverage
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/CompanyServiceTest.java#L223C15-L223C71>
Test updating the rating of the existing review business logic
5. (review service)
verifyReviewBelongsToCompany_whenMultipleReviewsForCompany_expectNoException
<https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ReviewServiceTest.java#L494C15-L494C91>
Testing review business logic

Integration testing

(Feature 1 + Feature 4)

reviewFlow_whenUserLogsInAndPostsReviewAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L84

Tests that an authenticated user can post a review for a company, integrating Authentication, Review Management, and Company Profile features, verifying the review is persisted and the company's average rating is updated correctly.

(Feature 1 + Feature 4)

reviewFlow_whenUserLogsInAndCreatesDuplicatesAndUpdatesReviewAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L154C7-L154C100

Tests that an authenticated user cannot submit duplicate reviews but can update an existing one, integrating Authentication, Review Management, and Company Profile features, verifying duplicate rejection, correct persistence, and accurate average rating recalculation.

(Feature 1 + Feature 2 + Feature 4)

wikiFlow_whenUserLogsInAndSearchesPaginatesReviewsAndAppliesAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/SearchPaginationCrossFeatureIntegrationTest.java#L95C7-L95C99

Tests that an authenticated user can search and paginate companies, leave a review, and create a job application from the company profile, integrating Authentication, Company Search, Review Management, and Application Management features together, verifying that all data is correctly persisted and consistent across features.

(Feature 1 + Feature 2 + Feature 4)

wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L96C7-L96C104

Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search, and Application Management features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

1. Given that I'm on the Company Wiki screen and want to create a new company entry, when I click the "Create A New Company Profile" button within the page, then I will be directed to the Create A New Company form to create a new company profile.

2. Given that I have completely filled in the form with valid information and the company has not yet been created, when I select the "Create" option, then I will be redirected to the Company Wiki screen and see the new company I added.
3. Given that I have completely filled in the form with valid information and the company has been created, when I try to create another company of the same name, then I should see a notification saying the company already exists and the company profile could not be created.
4. Given that I have incompletely filled in the form, when I select the "Create" option, then I should remain on the form and the missing fields should be marked in red.
5. Given that I'm on the Company Wiki screen, when I click on a company name, then I should be directed to the company profile.
6. Given that I'm looking at a company profile, when I look at the bottom of the page, then I should see a button to "Add A New Review".
7. Given that I'm looking at the "Add A New Review" button, when I click the button, then I should be directed to the form to add a new review.
8. Given that I have incompletely filled in the form, when I select the "Create" option, then I should remain on the form and the missing fields should be marked in red.
9. Given that I'm on the Company Wiki screen, when I click on a company name, then I should be directed to the company profile.
10. Given that I'm looking at a company profile and there are reviews that exist for that company, when I look at the reviews section, then I should be able to see the reviews about the company.
11. Given that I'm looking at a company profile and there are no reviews that exist for that company, when I look at the reviews section, then I should see a message saying that there's no reviews.
12. Given that I'm looking at the reviews about the company and there are more reviews that are not displayed on the page, when I look at the top-right corner, then I should be able to see a "next page" button.
13. Given that I click the next page button, when I look at the reviews section, then I should be able to see a new set of reviews.
14. Given that I've clicked the next page button and a previous page exists, when I look at the top-right corner, then I should be able to see a "previous page" button.
15. Given that I click the previous page button, when I look at the reviews section, then I should be able to see a previous set of reviews.

Test Level	Scope & Requirement	Methodology (How will you do this?)
Unit Testing	Feature 1: 173 tests Feature 2: 100 tests Feature 4: 317 tests ~200 tests per core feature. Total: 590 tests.	We use JUnit for automated unit testing There are three main components in the backend: Database, Service - containing business logic, and Controller - exposing endpoints to clients.

		For testing the database, we create a testing MongoDB instance of the database in containers (@Testcontainers) For service, we test business logic by testing the MongoDB instance of the database in containers (@Testcontainers). If the testing service interacts with other services, we mock other testing services. For the controller, each test will spin up a mock MVC instance and perform API calls to the mock MVC instance. If the testing controller interacts with other services, we mock other services.
Integration Testing	We have implemented 5 integration tests in total to cover interactions between the first 3 features we made and plan to do 5 more when we add the next 3 features. The code for the current 5 can be found here (one of the files contains 2 tests) and here is a brief description of 3 of the current integration tests: The CompanyReviewCrossFeatureIntegrationTests class contains 2 tests that validate review creation, updates, duplicate prevention, company average rating propagation, and authentication flows. The SearchPaginationCrossFeatureIntegrationTest class contains 1 test that covers searching	We use @SpringBootTest to load the full application context, @AutoConfigureMockMvc to enable realistic API endpoint testing via MockMvc, and @Testcontainers to spin up isolated MongoDB instances per test class. Each test simulates complete end-to-end user flows, such as logging in, searching or viewing companies, creating or updating reviews and applications, deleting resources, and logging out across features like authentication, companies, reviews, and applications. Tests assert HTTP responses, status codes, and JSON paths in the responses; they extract cookies and JWTs to maintain session continuity; and they directly verify persisted data, counts, and integrity in repositories such as companyRepository and reviewRepository.

	and paginating companies, followed by creating a review and an application. The <code>ApplicationLifecycleCrossFeatureIntegrationTest</code> class contains 1 test that verifies the full application lifecycle, including creation, updating, and deletion, while ensuring company and user data persistence.	Cross-feature integrity is ensured by asserting that counts in unrelated collections remain unchanged. All tests are executed automatically via GitHub Actions CI on every PR and push to the main and development branches, alongside unit tests, to provide comprehensive coverage.
Acceptance Testing	Feature 1: - Aidan 18 tests Feature 2: - Kate 5 tests Feature 4: - Nik 15 tests Total: 38 tests.	Acceptance testing has been done manually by walking through the acceptance criteria defined above. The walkthroughs, including screenshots, can be found in our frontend repository.
Regression Testing	Unit + Integration tests executed on every PR and push to the main and development branches.	We have configured a GitHub Actions CI pipeline to run all tests automatically. We also configured GitHub Actions CI pipeline (Codecov) to generate a code coverage report for every PR to the main and dev branches.

2.1.1 CI/CD Regression Workflow

Briefly describe your automated pipeline.

- Frontend
 - Every commit to both the develop and main branches triggers the CI pipeline running build and lint steps
 - Every pull request with a target branch of main or develop triggers a CI pipeline running build and lint steps. The pipeline must pass before the PR can be merged
- Backend
 - Each commit to either main or dev will trigger
 - A build of the Docker image
 - A Java code formatting check
 - A Gradle build of the backend
 - Running unit and integration tests
 - Codecov test coverage checking

- Each pull request for the branches main or dev will trigger
 - A build of the Docker image
 - A Java code formatting check
 - A Gradle build of the backend
 - Running unit and integration tests
 - Codecov test coverage checking
- CD can be triggered manually to build and push the Docker image to DockerHub

2.3 Mutation Testing (Test effectiveness)

Skip for sprint 2.

2.4 Load Testing

Skip for sprint 2.

3 Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Programming Interface
AUT	Application Under Test
dev	Development
prod	Production